

PB007: Software Engineering

Lecture 1: Introduction

Stanislav Chren

Autumn 2026

Outline

1. Course Organisation
2. Software Engineering Introduction
3. System Modelling

Slido.com
#PB007



1. Course Organisation

About the lecturer: Stanislav Chren



Research

- PhD at Lasaris, FI MU
 - Software quality analysis
 - Reliability of critical infrastructures
- Postdoc at Aalto University, Finland
 - Data-driven software development
 - Continuous experimentation

Teaching

- PB007 Software engineering (FI MU)
- PV260 Software quality (FI MU)
- Software Architectures (Aalto)
- Software Testing and QA (Aalto)

Industry

- Software engineer
 - Projects in energy domain, smart grids
 - Collaboration with ČEZ, Mycroft Mind, ...

About the course: PB007 Software Engineering I

Lectures

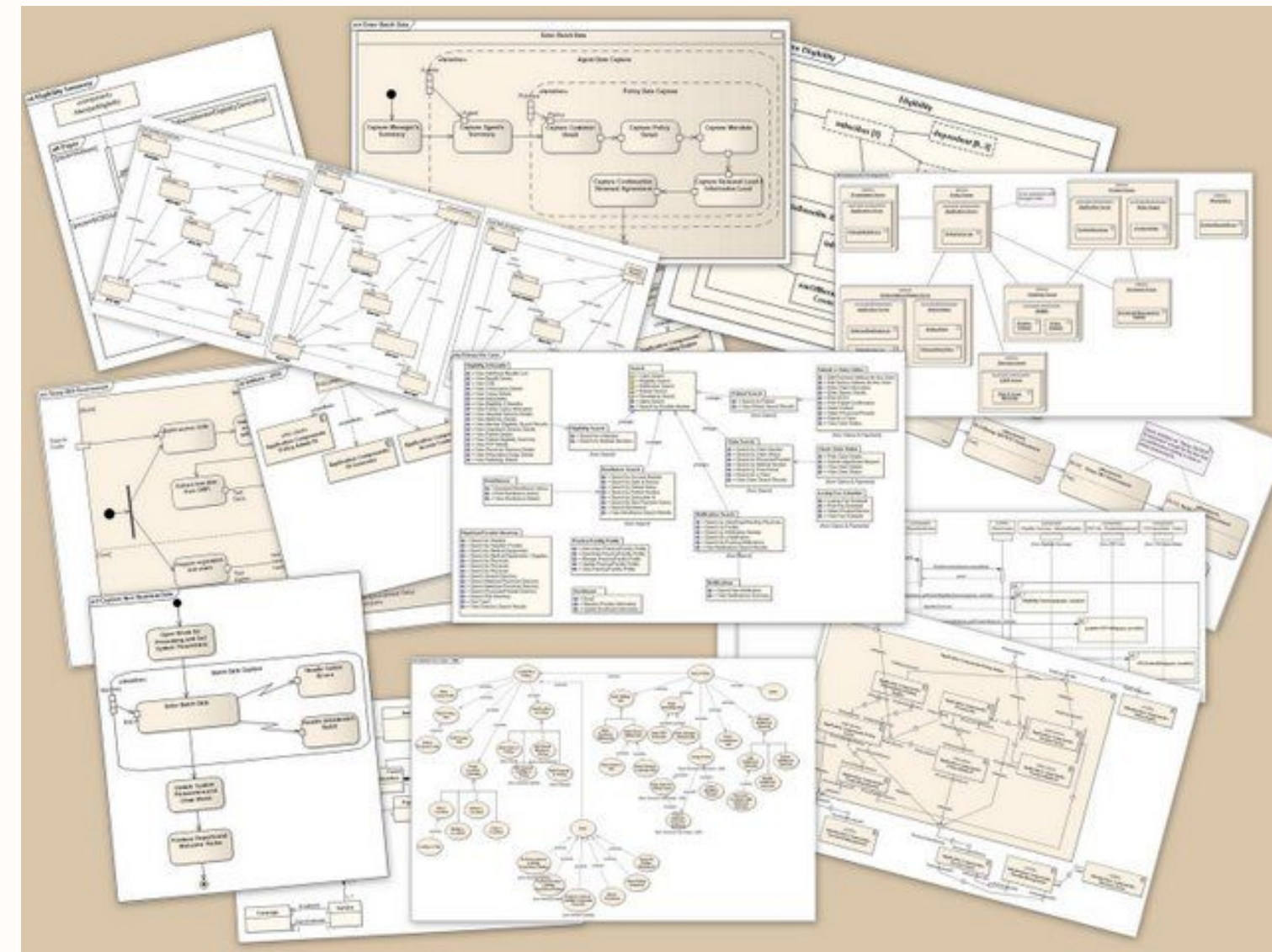
1. Software engineering fundamentals, **Use Case diagram**.
2. Requirements specification, **UML Activity diagram**.
3. System analysis and design, **UML Class and Object diagram**.
4. Agile and other methods.
5. Data modelling and management, **ERD**.
6. High-level design, **UML Class diagram in design, Package diagram**.
7. Low-level design and implementation, **UML Interaction diagrams**.
8. Architecture design, **UML Component and Deployment diagrams**.
9. Testing, verification and validation, **UML State diagram**.
10. Operation, maintenance and system evolution.
11. Software development management.
12. Advanced software engineering techniques.
13. *TBA*
14. *TBA*



About the course: PB007 Software Engineering I

Seminars

1. Tool Introduction, project assignment
2. Initial Use Case Diagram
3. Detailed Use Case Diagram, textual specification
4. Graphical specification of use cases: Activity Diagram
5. Analysis Class Diagram, domain modelling
6. Data Modelling, Entity Relationship Diagram (ERD)
7. Finalisation of Analysis Class Diagram
8. Design Class Diagram, interfaces, implementation Details
9. Design Class Diagram finalization, Package Diagram,
10. Interaction Diagrams (Sequence)
11. State Diagram
12. Finalisation of Sequence and State Diagram
13. Deployment Diagram
14. Project Review



About the course: PB007 Software Engineering I

Prerequisites

- Basic knowledge of object oriented programming

Lectures

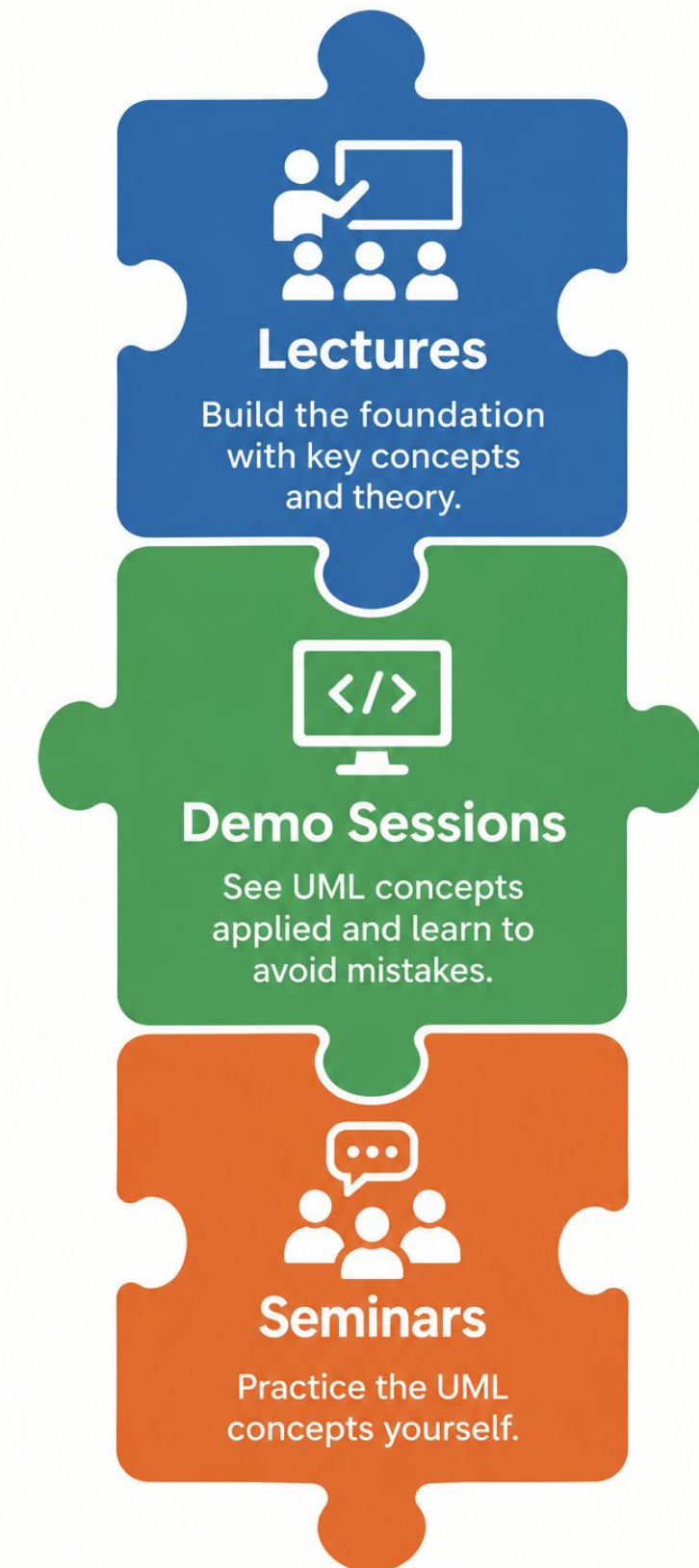
- 12 + 2 teaching weeks
- Streamed and recorded

Demo sessions

- ~10 sessions
- Streamed and recorded

Seminars

- 13 + 1 teaching weeks
- Team project on UML modelling
 - teams of 3 students
- Mandatory attendance
 - one absence ok
 - extra absences (-5 points)
- Weekly assignment submission
 - assignment grading (0-2 points)
 - late assignment submission (-5 points)



About the course: PB007 Software Engineering I

Evaluation

- **Seminar**

- **project YES/NO**
- **assignment points (0-20 points)**
- **bonus activity points (0-5 points)**
- **penalties**

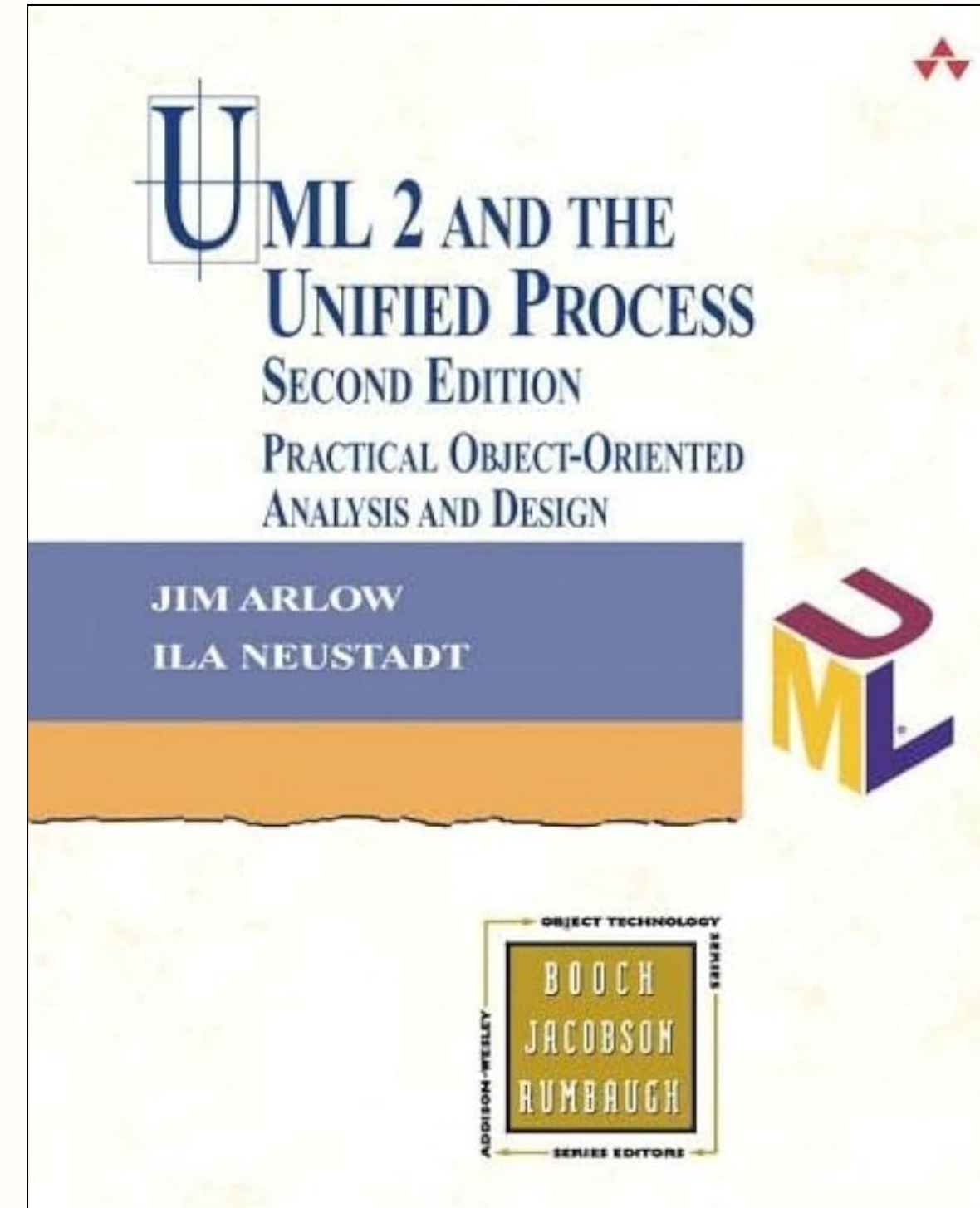
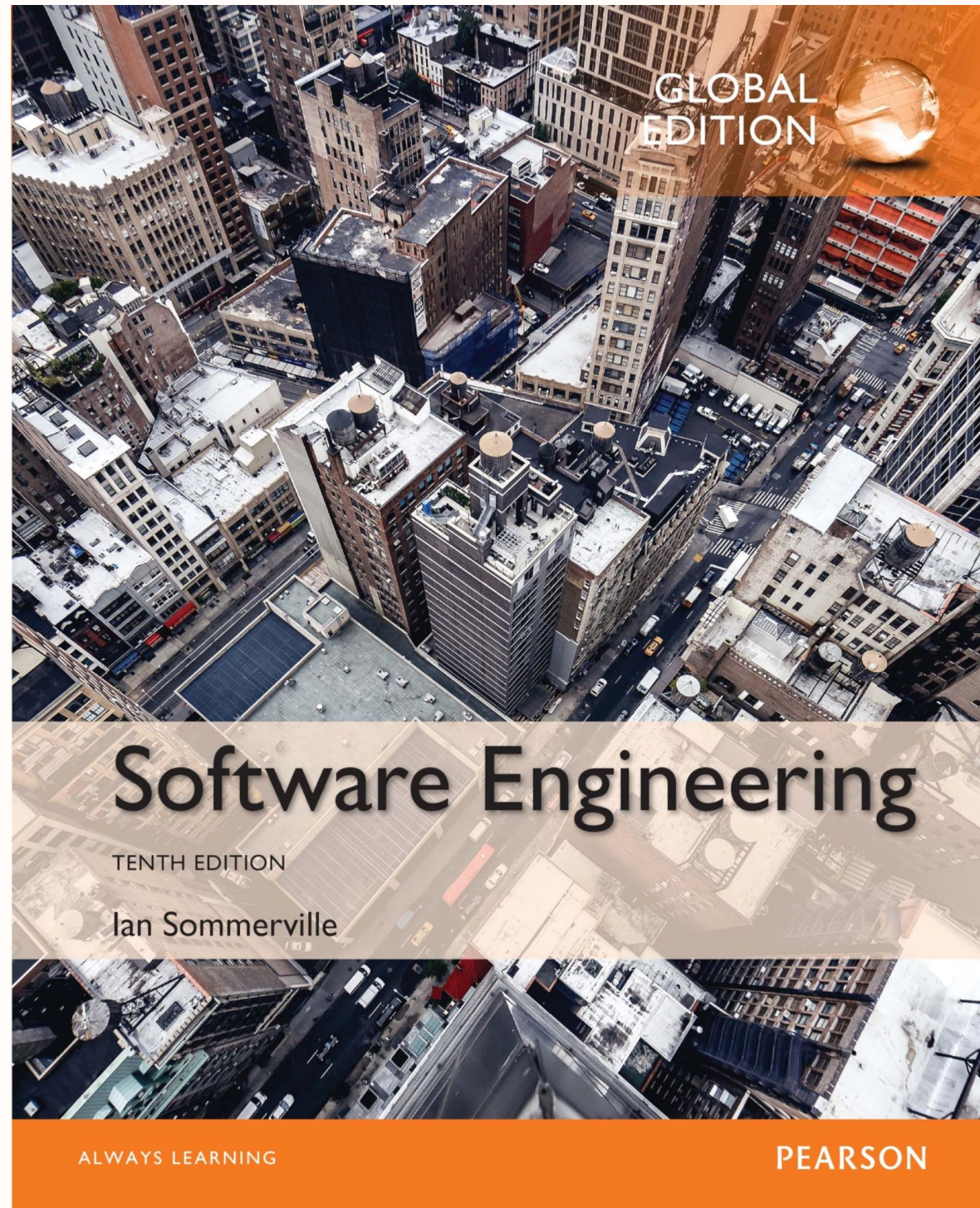
- **Exam**

- **test (35 points)**
- **on-site modelling (35 points)**
 - **minimum 15 points required**

Points	Grade
82+	A
74-81	B
66-73	C
58-65	D
50-57	E
<50	F

About the course: PB007 Software Engineering I

Study materials



Questions?

- **Submit them to the course Discord server**
- **Contact your seminar tutors**
 - Questions/issues about project, tasks, teamwork
 - Seminar group channels on Discord, DMs
 - Emails
- **Contact the lecturer**
 - Questions/issues about UML, course, exams, ...
 - Email: chren@mail.muni.cz, Discord
 - Consultations
 - Fridays 14:00-15:00
 - Office C330 (in person) or remote
 - Best to arrange in advance



2. Software Engineering Intro

Software is Everywhere!

- Business, government, finance, health care
- Communications, energy, water, logistics
- Devices, appliances
- Machines, transportation
- AI systems

If software stopped, much of society would stop



Software is Expensive!

- The cost of software development often outweighs that of hardware
- Large software projects can cost hundreds of millions
- Examples
 - e-Legislation - ~1000 M CZK
 - Agricultural Intervention Fund - ~500 M CZK/year
 - Apotti system in Finland ~800 M EUR

Software engineering aims to develop software cost-efficiently



Software Failures can be Catastrophic!

- **Human lives can be lost**
 - **Therac-25, Tesla**
- **Careers can be lost**
 - **Volkswagen CEO Herbert Diess**
- **Money can be lost**
 - **FN Brno ransomware in 2020 - ~100s M CZK**
 - **Knight Capital Group losing \$440M in 30 minutes due to a deployment error**

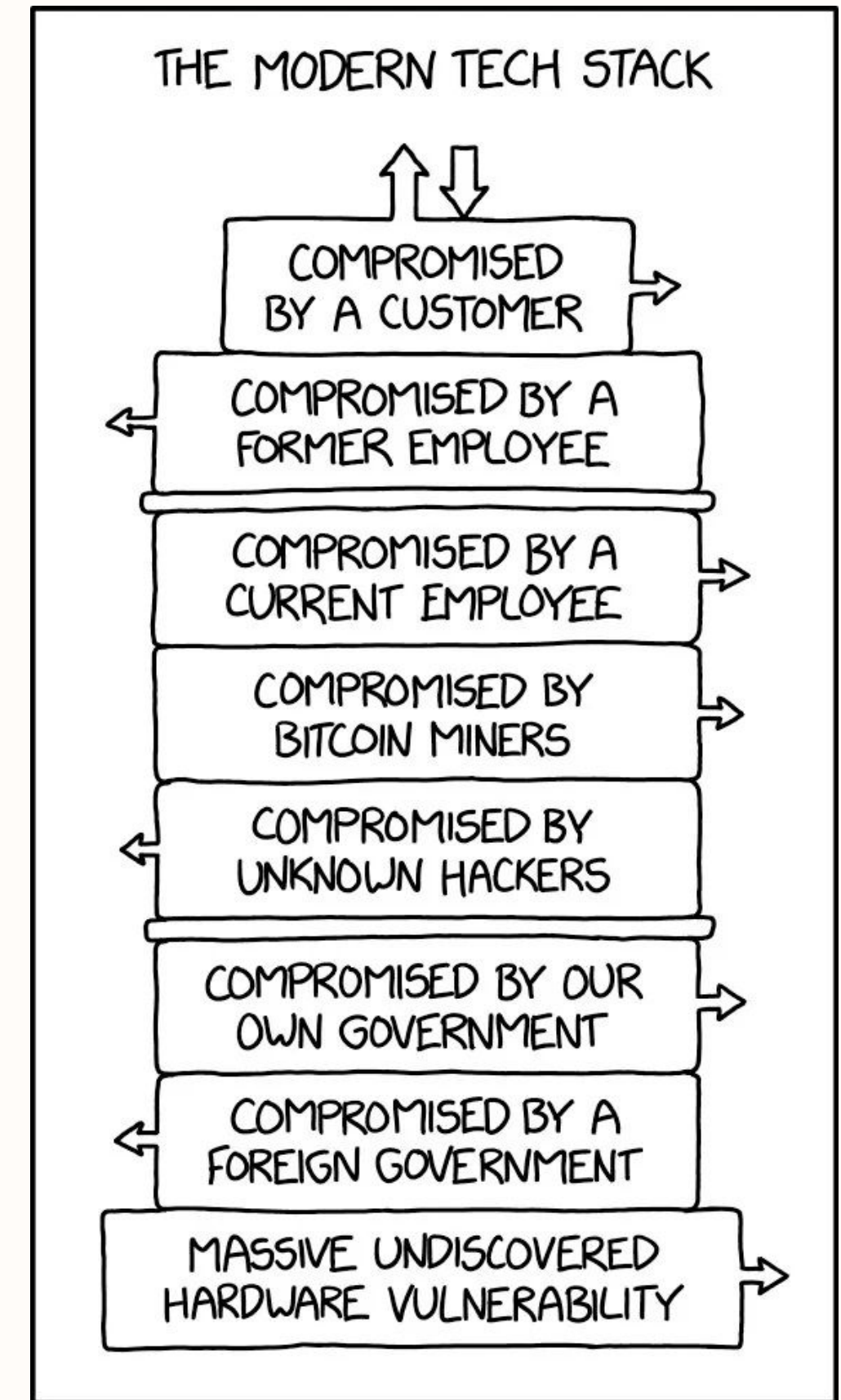


Software engineering is about creating working and reliable software given the real-world constraints

Why Software Fails?

- Software failures are persistent, common and expensive
- Failures often arise from:
 - unclear or changing requirements
 - unrealistic expectations
 - poor communication
 - weak risk management
 - organisational and management problems

Software failures are often socio-technical



xkcd 2166

Historical Starting Point

- **1968 and 1969: NATO Software Engineering Conferences**
- **Large software projects were late, over budget, unreliable and hard to maintain**
- **The term software engineering emerged as a response**
- **The goal: bring more systematic, professional, engineering-like discipline to software work**

Software projects became too large, too interconnected, and too consequential to depend only on individual skill and improvisation



**Working group meeting at the NATO Software Engineering Conference,
Garmisch, Germany, 1968**

**Software systems
must be
professionally
developed, operated, and maintained**

Software Engineering Definition

“The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software”

IEEE Std. 610.12–1990 and ISO/IEC/IEEE Vocabulary

Software Engineering Definition

“The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software”

IEEE Std. 610.12–1990 and ISO/IEC/IEEE Vocabulary

Systematic
Intentional, repeatable ways of working

Disciplined
Agreed practices followed by teams

Quantifiable
Evidence, measurement, feedback

Development, operation, maintenance
The whole lifecycle



Engineering

What Counts as Software?

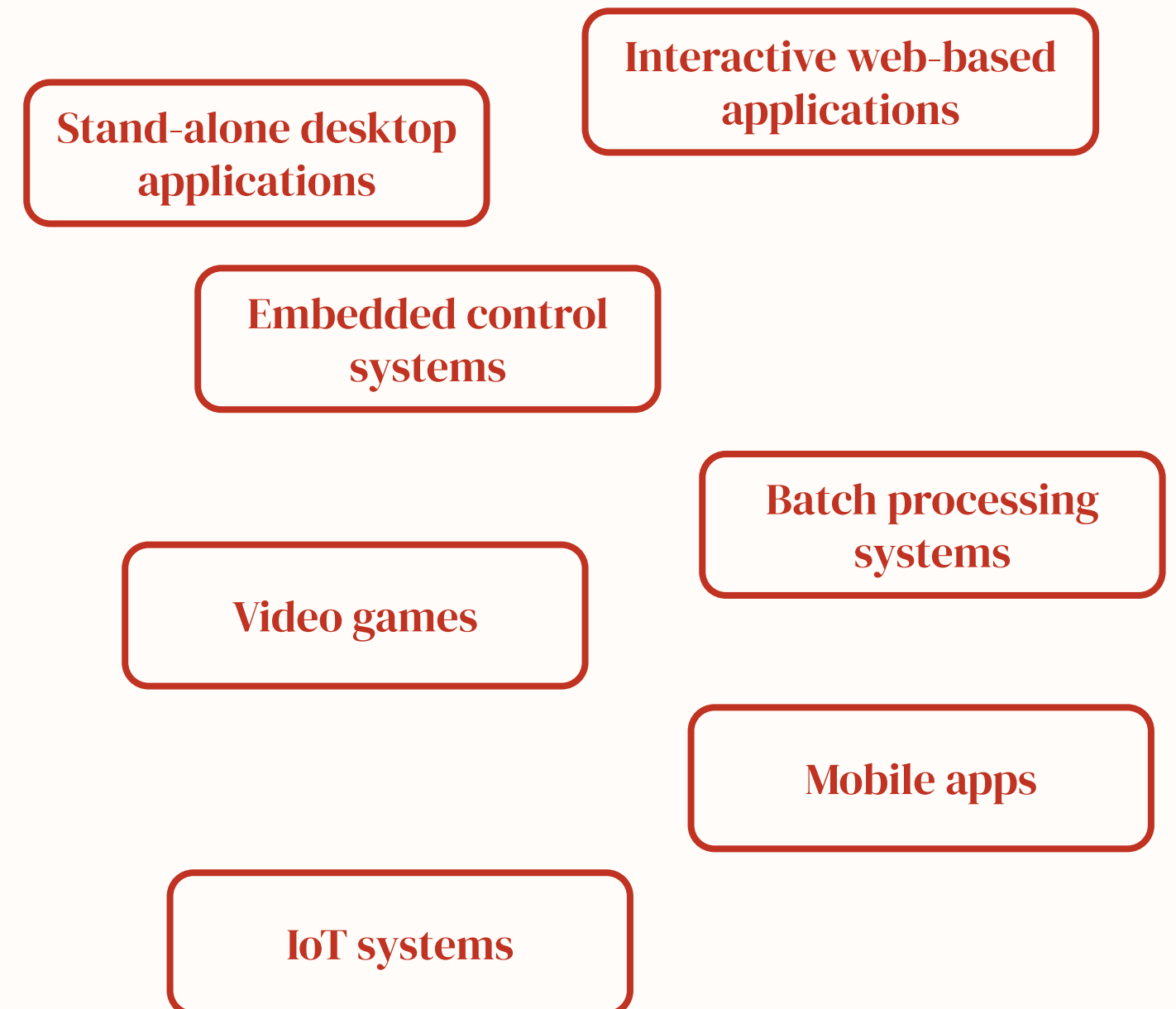
- **Source code is only one part**
- **Software engineering also concerns:**
 - Requirements and specifications
 - Architecture and design documents
 - Tests and test data
 - Deployment and configuration
 - Operational procedures
 - User and maintainer documentation
 - The whole product/service in use as a software system

Software engineering manages the artefacts needed to build, operate, and evolve the system while considering contextual factors and constraints.

Software products



Application types



Software Engineering Body of Knowledge – SWEBOK

- Promotes a consistent worldwide view of software engineering
- Defines 18 knowledge areas, specifying the scope of software engineering
- Characterizes the contents of software engineering within each knowledge area
 - References to other materials
- Latest official version: V4 from 2024

#	Knowledge Area
1	Software requirements
2	Software architecture
3	Software design
4	Software construction
5	Software testing
6	Software engineering operations
7	Software maintenance
8	Software configuration management
9	Software engineering management
10	Software engineering process
11	Software engineering models and methods
12	Software quality
13	Software security
14	Software engineering professional practice
15	Software engineering economics
16	Computing foundations
17	Mathematical foundations
18	Engineering foundations

Software Engineering Body of Knowledge – SWEBOK

Understand the problem and shape the solution

- Are we building the right thing, and can it be built well?

Requirements

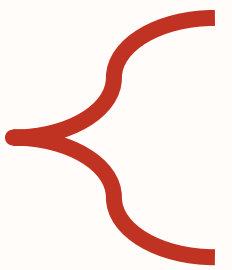
- Understanding and management of what should be built, for whom and under what constraints

Architecture

- Making high-level structural and quality decisions and trade-offs

Design

- Definition of interfaces, components, data and interactions



#	Knowledge Area
1	Software requirements
2	Software architecture
3	Software design
4	Software construction
5	Software testing
6	Software engineering operations
7	Software maintenance
8	Software configuration management
9	Software engineering management
10	Software engineering process
11	Software engineering models and methods
12	Software quality
13	Software security
14	Software engineering professional practice
15	Software engineering economics
16	Computing foundations
17	Mathematical foundations
18	Engineering foundations

Software Engineering Body of Knowledge – SWEBOK

Create and check the software

- Have we built the software right?
- What evidence do we have to assess the risks?

Construction

- Coding, debugging, integration

Testing

- Dynamic validation on selected test cases

Quality

- Quality management, assurance and improvement

Security

- Engineering secure systems from the start.

#	Knowledge Area
1	Software requirements
2	Software architecture
3	Software design
4	Software construction
5	Software testing
6	Software engineering operations
7	Software maintenance
8	Software configuration management
9	Software engineering management
10	Software engineering process
11	Software engineering models and methods
12	Software quality
13	Software security
14	Software engineering professional practice
15	Software engineering economics
16	Computing foundations
17	Mathematical foundations
18	Engineering foundations

Software Engineering Body of Knowledge – SWEBOK

Run and evolve the system

- How do we keep the system running smoothly?
- How do we keep the system in shape?

Operations

- Deployment, monitoring, control, support

Maintenance

- Fix defects and adapt to changing needs

Configuration management

- Version control, change control, releases

#	Knowledge Area
1	Software requirements
2	Software architecture
3	Software design
4	Software construction
5	Software testing
6	Software engineering operations
7	Software maintenance
8	Software configuration management
9	Software engineering management
10	Software engineering process
11	Software engineering models and methods
12	Software quality
13	Software security
14	Software engineering professional practice
15	Software engineering economics
16	Computing foundations
17	Mathematical foundations
18	Engineering foundations

Software Engineering Body of Knowledge - SWEBOK

Organise the work

- How do we design and manage the organisation that performs the work?

Management

- Plan, estimate, coordinate, measure and lead

Process

- Lifecycle models, assessment and improvement

Models and methods

- Modelling, analysis, agile and other methods

Economics

- Value, costs, estimation and trade-off decisions

#	Knowledge Area
1	Software requirements
2	Software architecture
3	Software design
4	Software construction
5	Software testing
6	Software engineering operations
7	Software maintenance
8	Software configuration management
9	Software engineering management
10	Software engineering process
11	Software engineering models and methods
12	Software quality
13	Software security
14	Software engineering professional practice
15	Software engineering economics
16	Computing foundations
17	Mathematical foundations
18	Engineering foundations

Software Engineering Body of Knowledge – SWEBOK

Professional and foundational knowledge

- What knowledge is required by our work?
- Software engineering is a profession, not only technical activity

Professional practice

- Ethics, responsibility, teamwork, communication

Computing foundations

- Core computing knowledge for engineers

Mathematical foundations

- Reasoning, models, probability, logic, and related tools

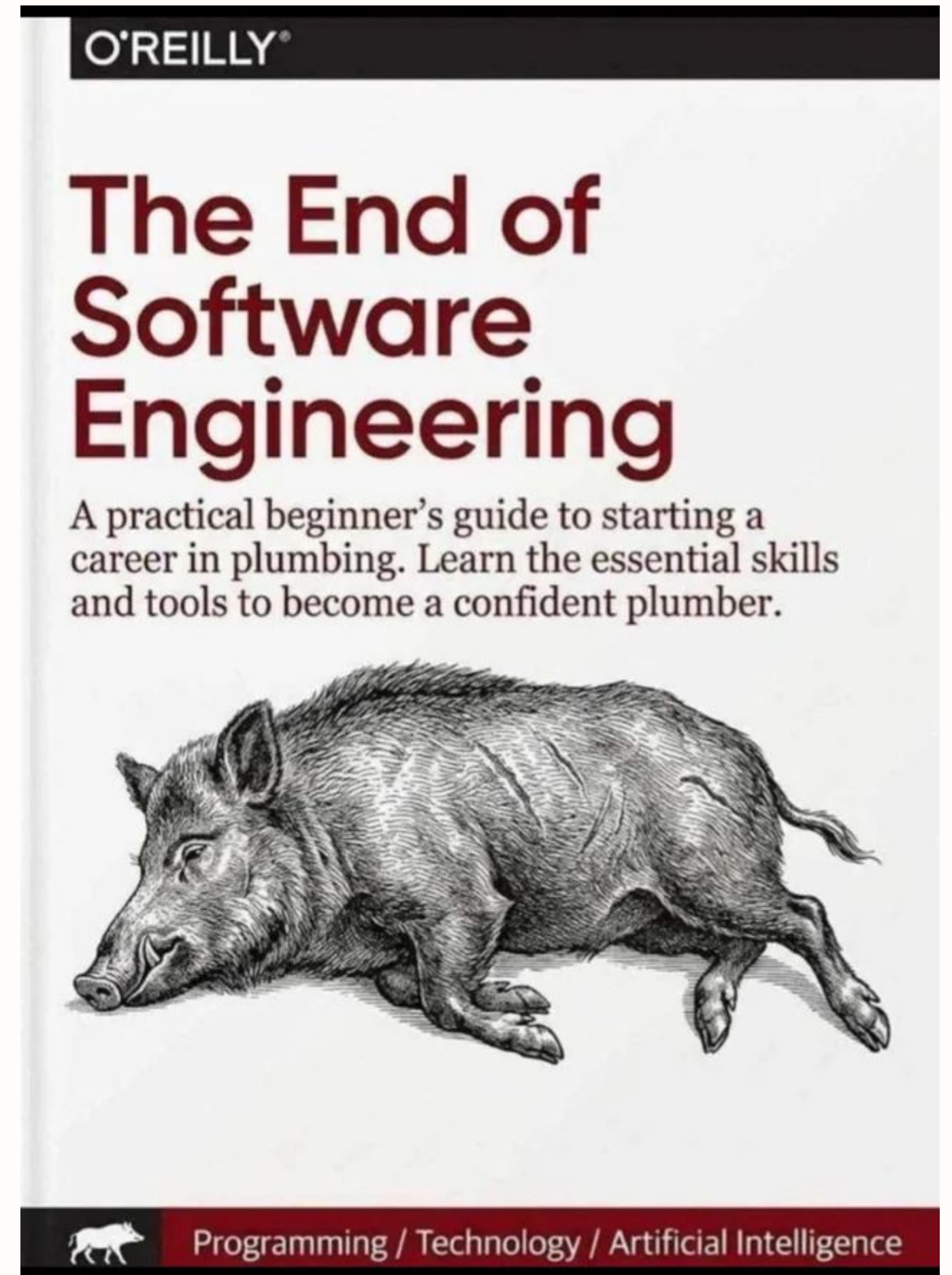
Engineering foundations

- General engineering principles and trade-offs

#	Knowledge Area
1	Software requirements
2	Software architecture
3	Software design
4	Software construction
5	Software testing
6	Software engineering operations
7	Software maintenance
8	Software configuration management
9	Software engineering management
10	Software engineering process
11	Software engineering models and methods
12	Software quality
13	Software security
14	Software engineering professional practice
15	Software engineering economics
16	Computing foundations
17	Mathematical foundations
18	Engineering foundations

Software Engineering and AI

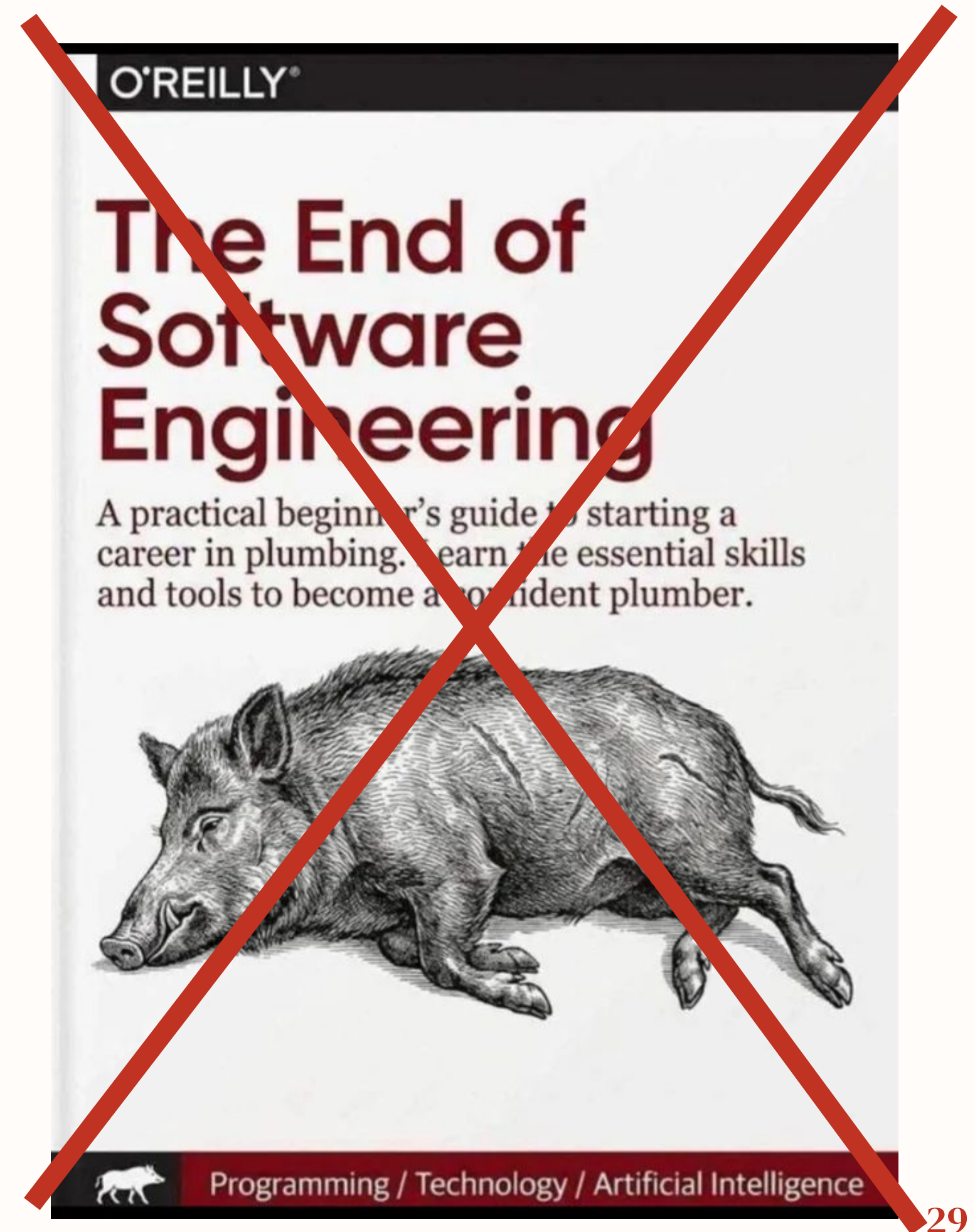
Are you worried about your software engineering career



Software Engineering and AI

- **AI for software engineering**
 - Code generation, code explanation test generation
 - Documentation
 - Design exploration
- **Software engineering for AI**
 - Data and model quality
 - Evaluation
 - Monitoring and drift
 - Security and privacy
 - Accountability

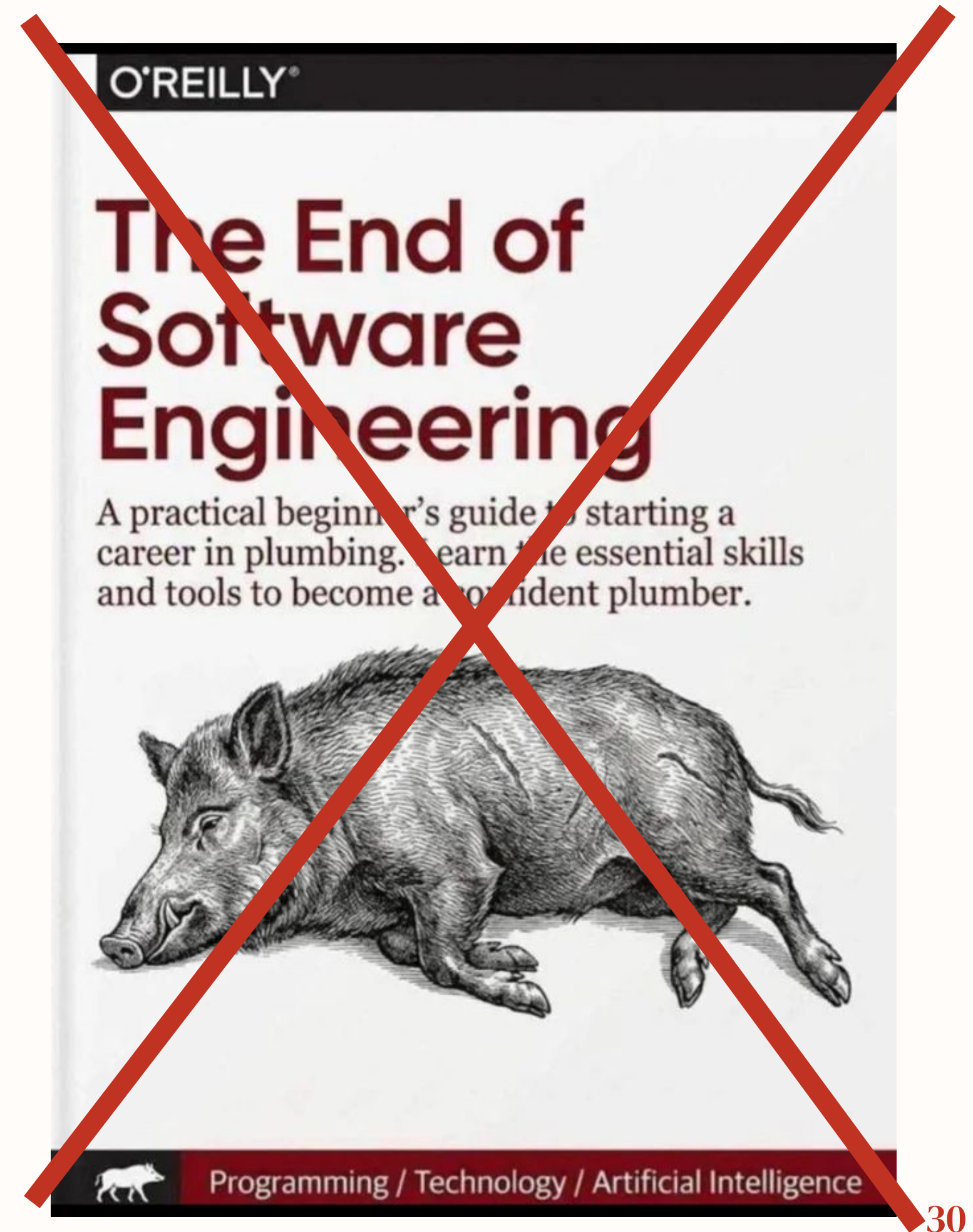
AI does not remove engineering; it changes where engineering judgement is needed.



Software Engineering and AI Challenges

- Correctness of the generated code
- Security vulnerabilities, licensing risks
- Unpredictable behaviour outside evaluation data
- Biased outputs, hard to explain, sensitive to prompts
- Responsibility remains with the people and organisations

AI makes validation, review, monitoring and accountability more important



Why is it Difficult to Create a Good Software?

“There is no single technique or tool that will make software development dramatically easy.”

Brooks, Frederick, and H. Kugler. "No silver bullet."
High-Integrity System Specification and Design (1987)



Accidental difficulties

- **Caused by tools and circumstances**

Essential difficulties

- **Caused by the nature of software itself**

Complexity

Many interacting states, rules, and exceptions

Conformity

Must fit human institutions and legacy constraints

Changeability

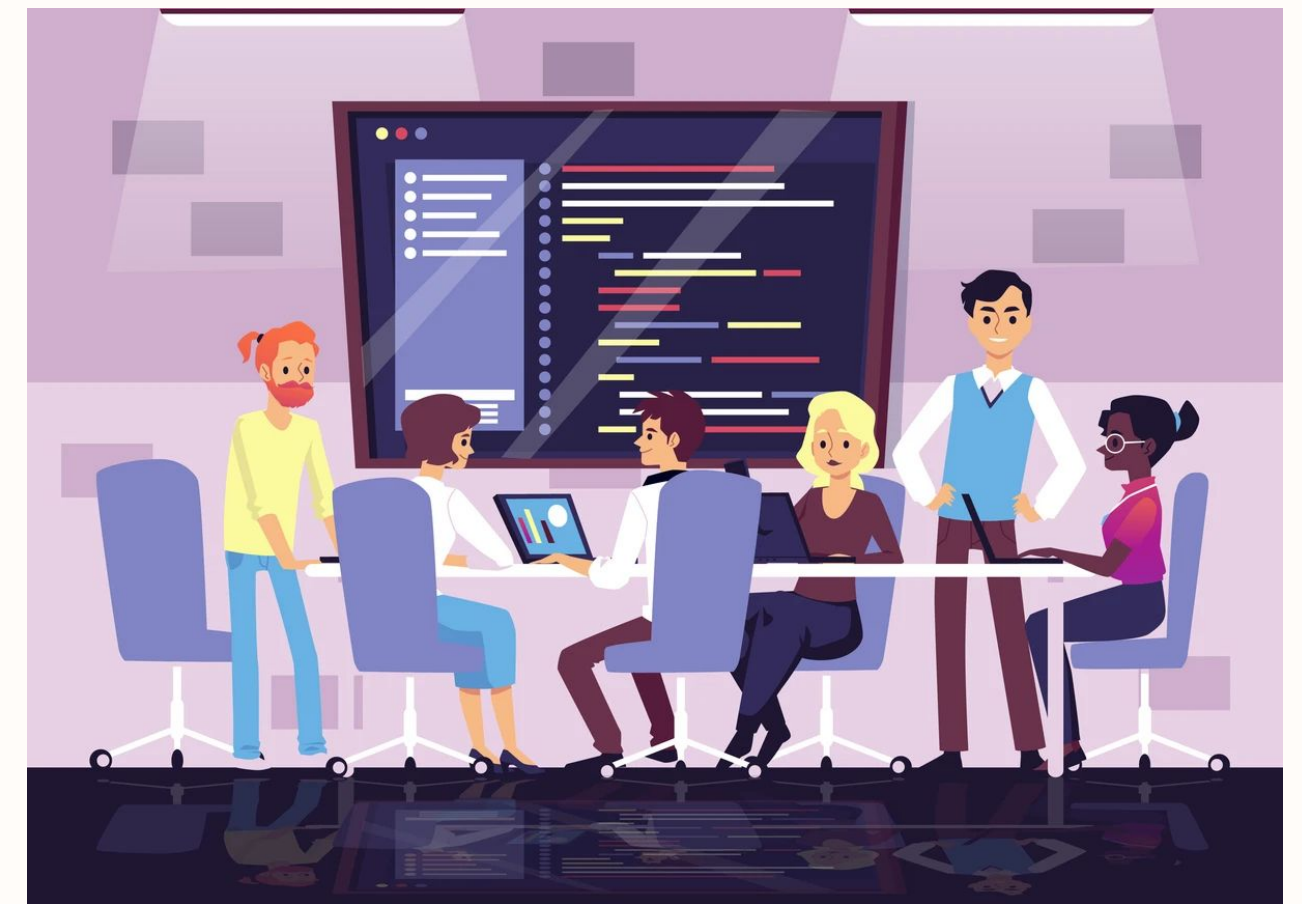
Expected to evolve as needs change over time

Invisibility

No natural physical form; hard to visualize fully

Software Engineering as a Career

- Studying software engineering supports working in many roles
 - Requirements engineer, developer, tester, architect,
 - Product owner, Scrum master, team lead
 - DevOps engineer, engineering manager
 - Entrepreneur
- The work is technical, human, organisational, requires domain knowledge and ethical understanding
- The goal is not just to ship code but to **create dependable value**



Summary

Software engineering exists because software became too important to build carelessly.

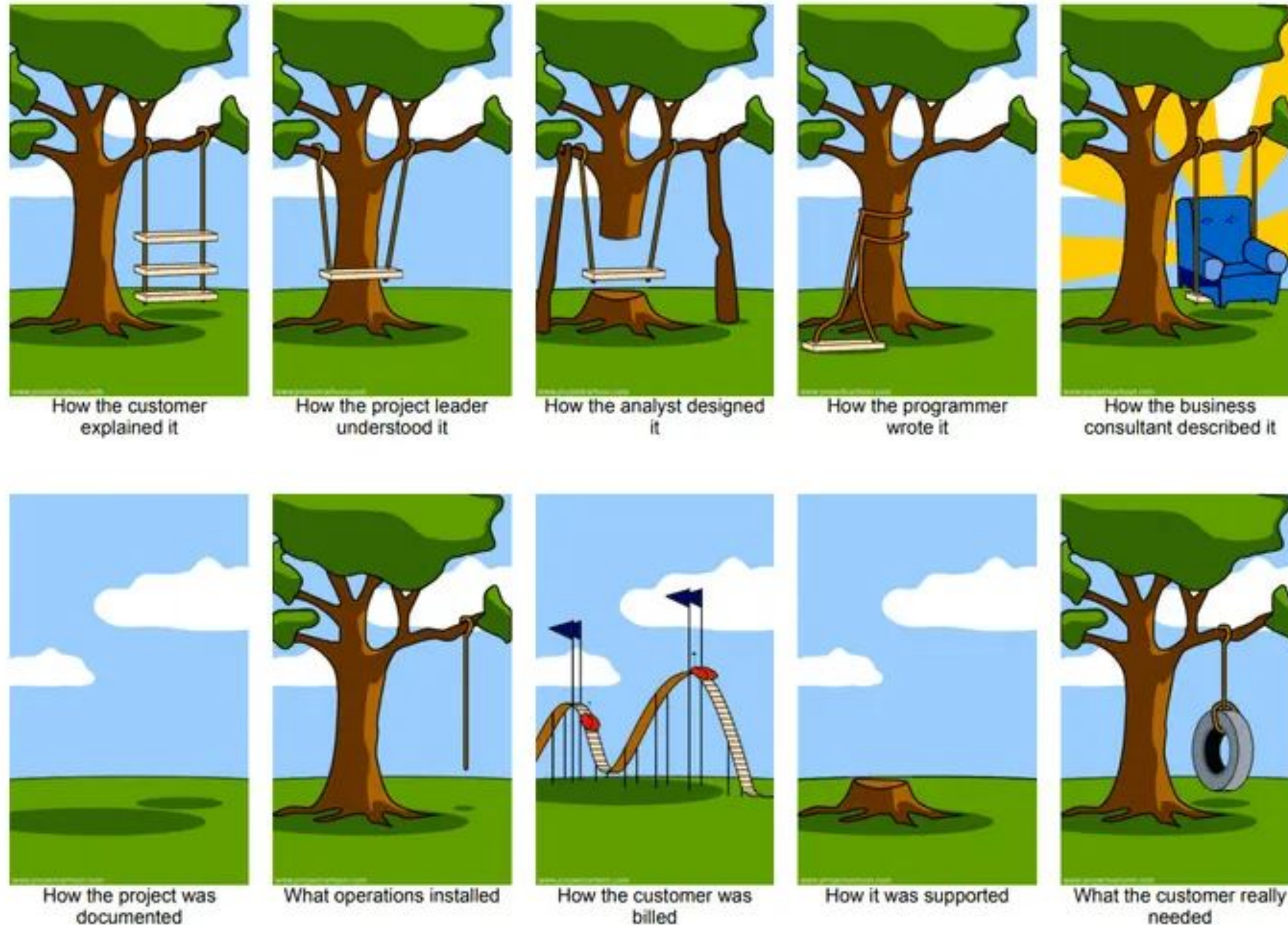
Technical solutions change. The engineering challenge remains:

Build software that delivers value, works dependably, can evolve safely, and deserves to be trusted

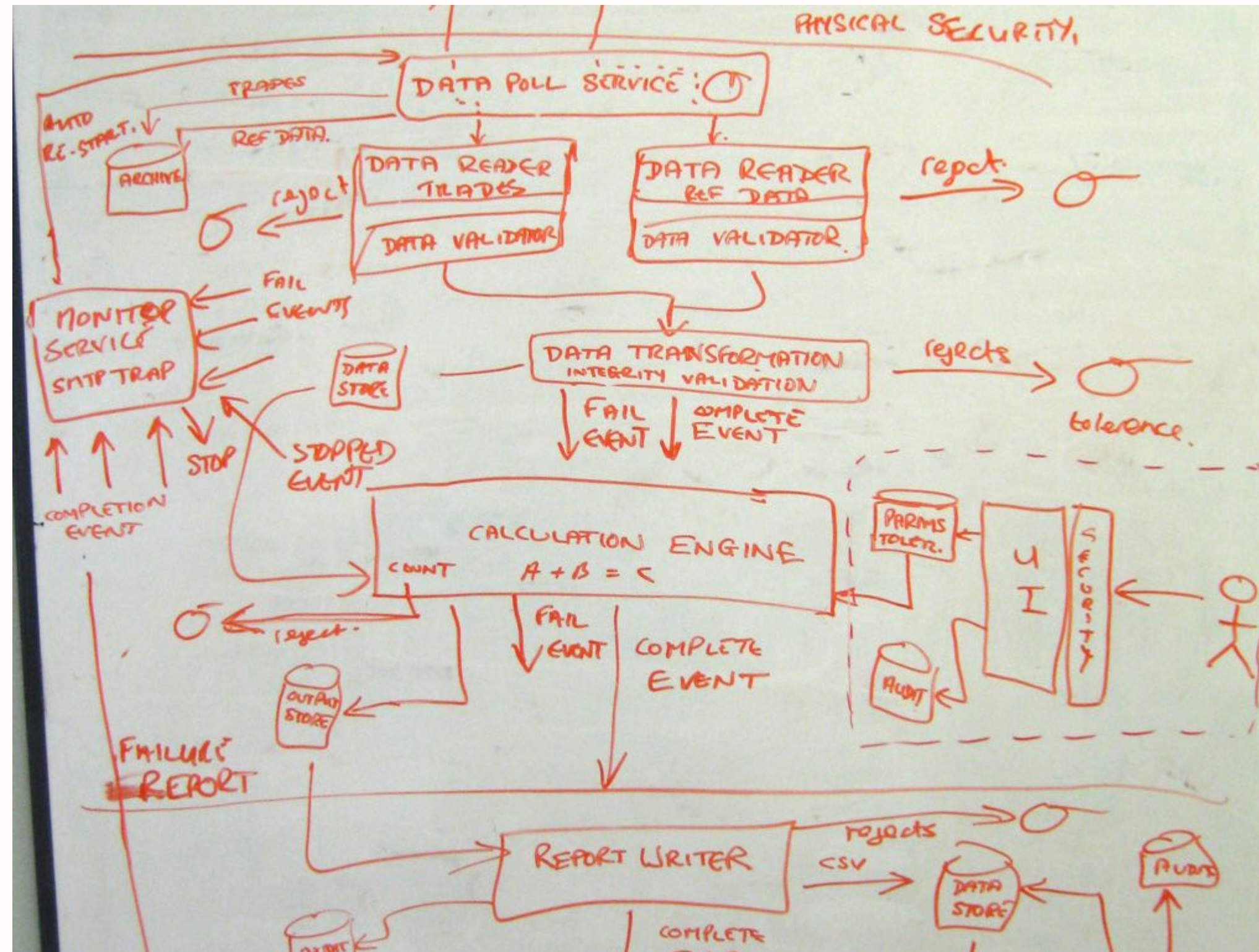
3. System Modelling

Why do we Need to Model Software?

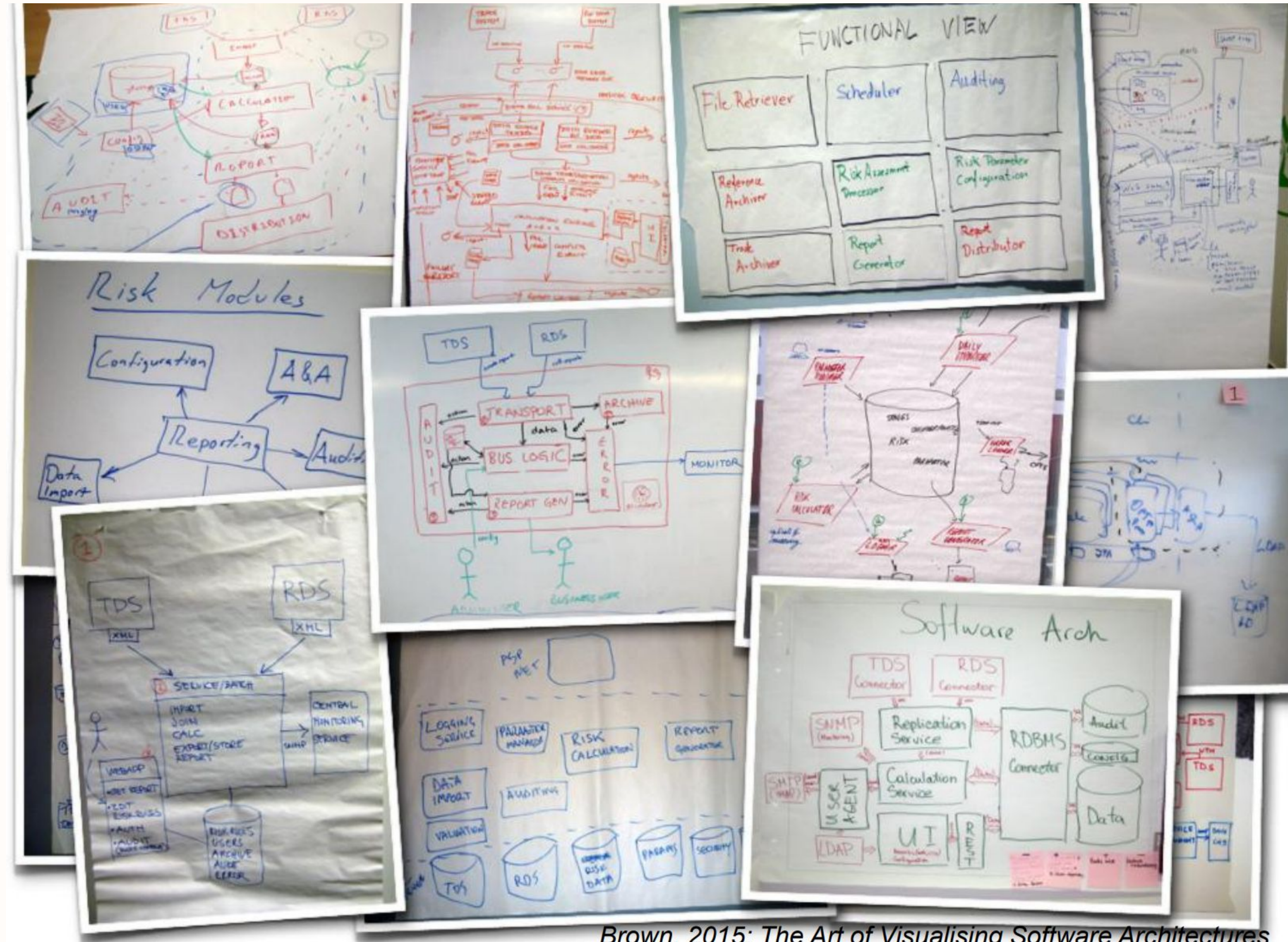
Why do we Need to Model Software?



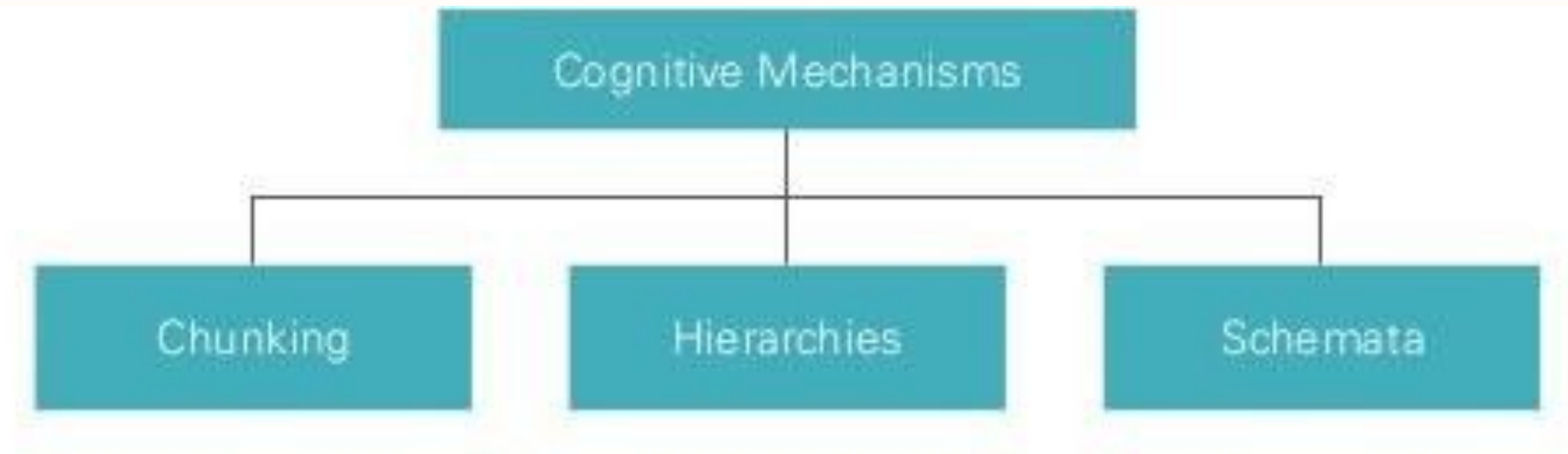
Why do we Need to Model Software?



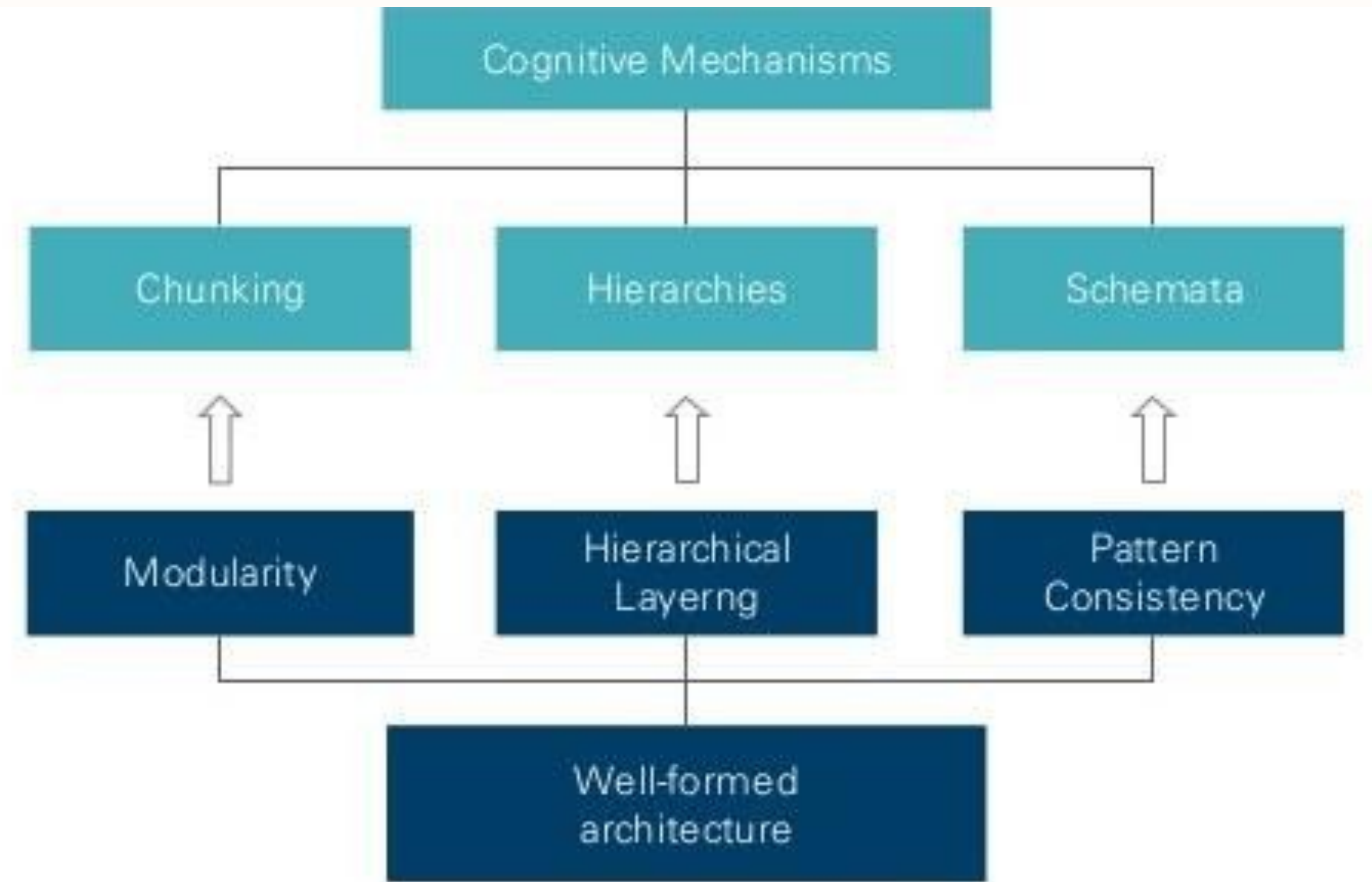
Why do we Need to Model Software?



Visuals Facilitate Understanding

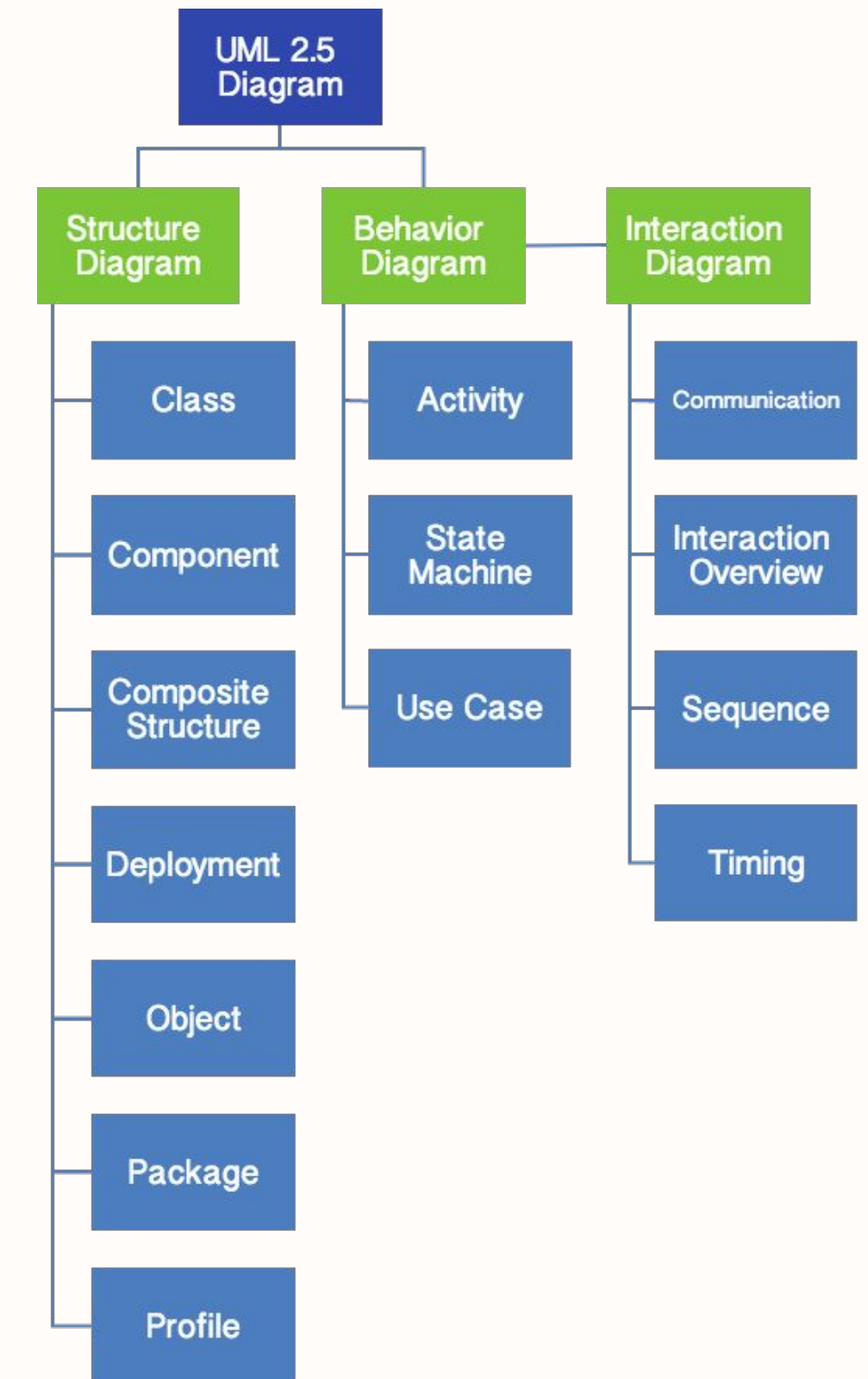
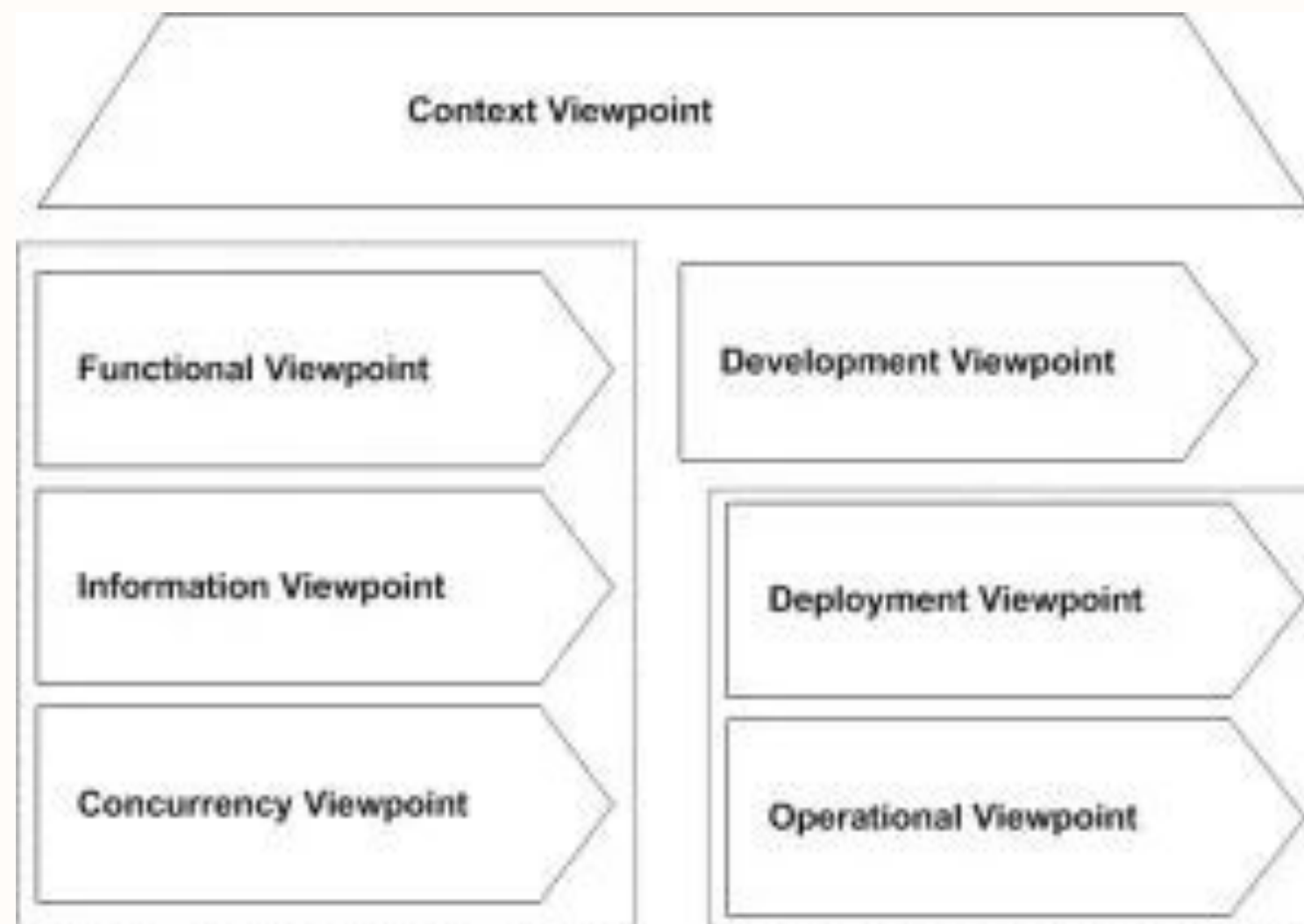


Visuals Facilitate Understanding



System Modelling

- **System modelling is the process of developing abstract models of a system.**
 - **Models present different views or perspectives of that system**



Unified Modeling Language (UML)

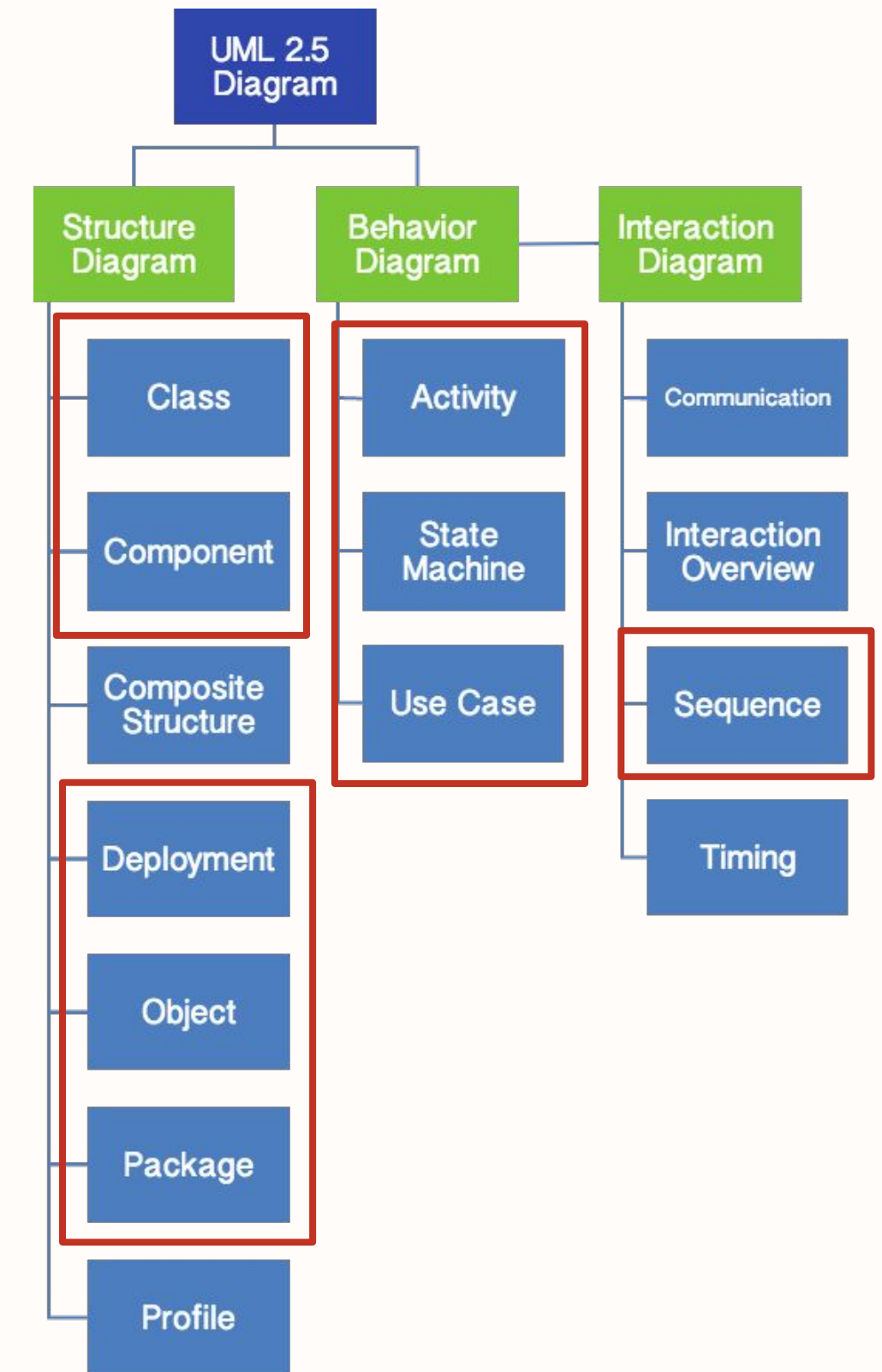
- Standardized visual language for modelling software systems.
- Focus on detailed analysis and design

Advantages

- Mature and well-known
- Standardised notation
- Expressive
- Tool support

Disadvantages

- Complex notation
- Less explicit connections between diagrams
- Too technical for non-developers



SysML

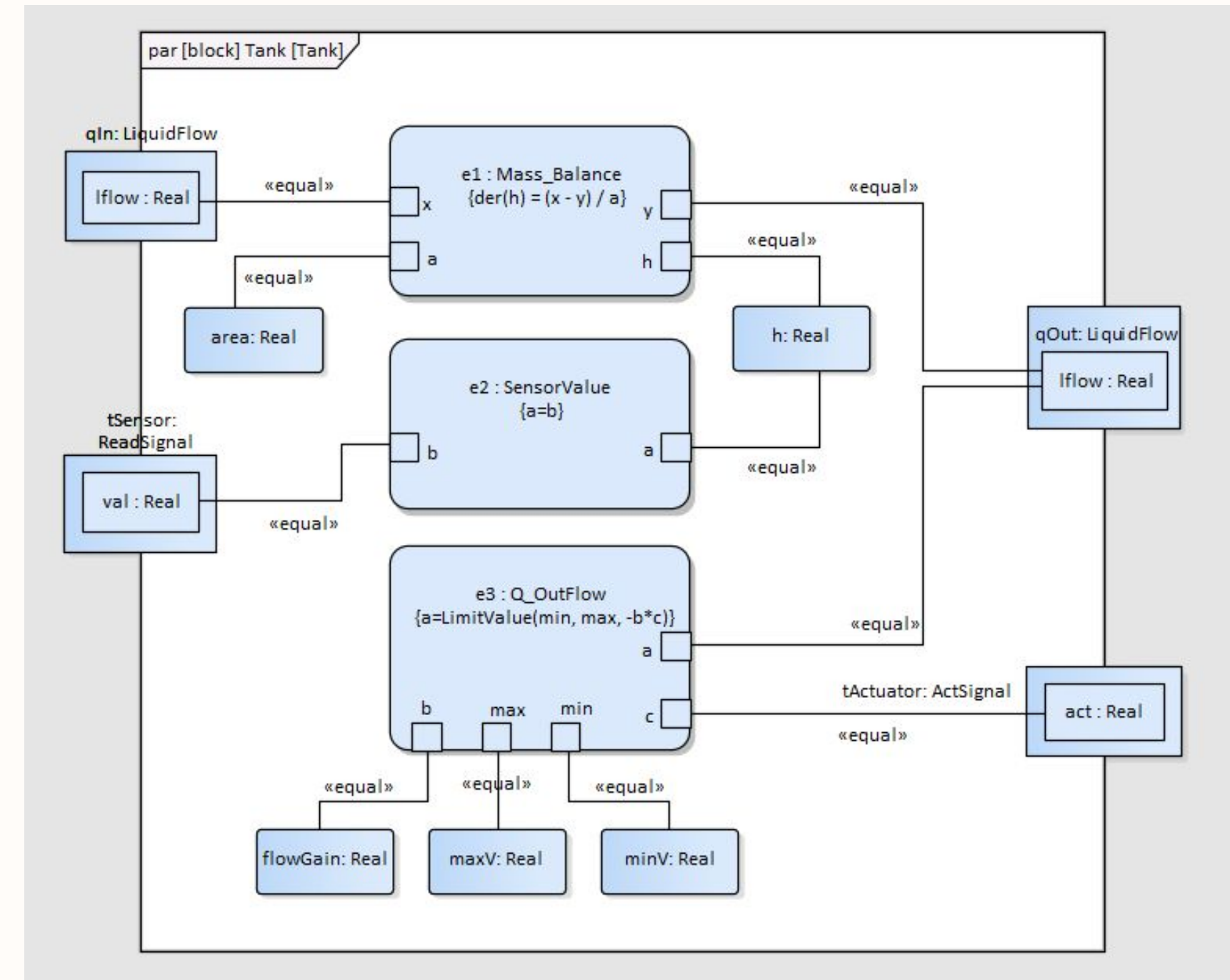
- Focus on modelling of cyber-physical systems
- Extension of UML
- Additional parameters for model execution

Advantages

- Covers hardware, software and people
- Better traceability of requirements
- Better system functional decomposition
- Integration with other engineering models

Disadvantages

- More complex than UML
- Difficult to learn
- Heavyweight tools



C4 Model

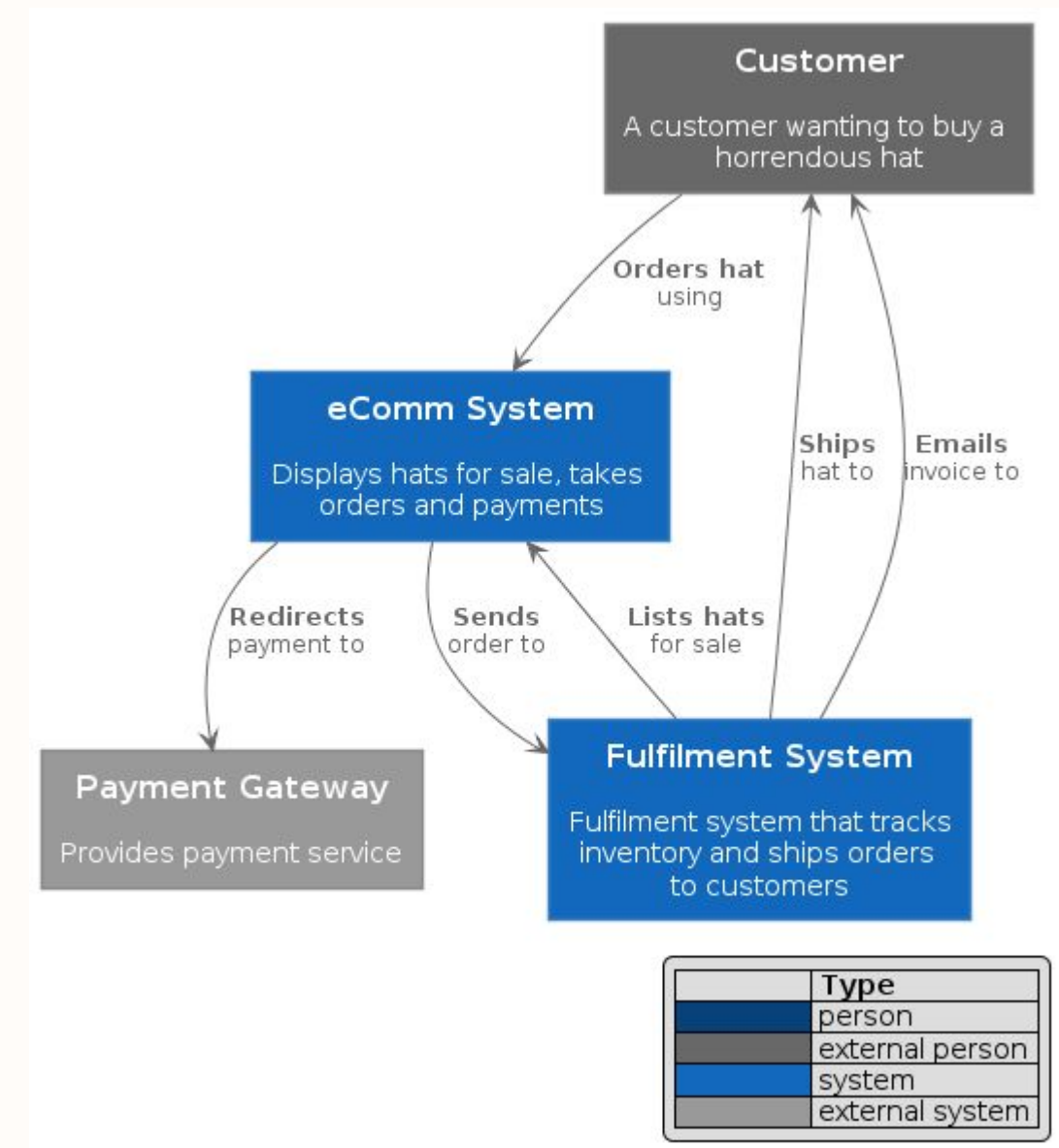
- Context, Containers, Components, Code
- Designed for wider understandability and communication

Advantages

- Easy to learn
- Explicit levels of abstraction
- Lightweight
- Good for agile development

Disadvantages

- Missing many types of models (e.g. for behaviour modelling)
- Less formal specification/semantics



ArchiMate

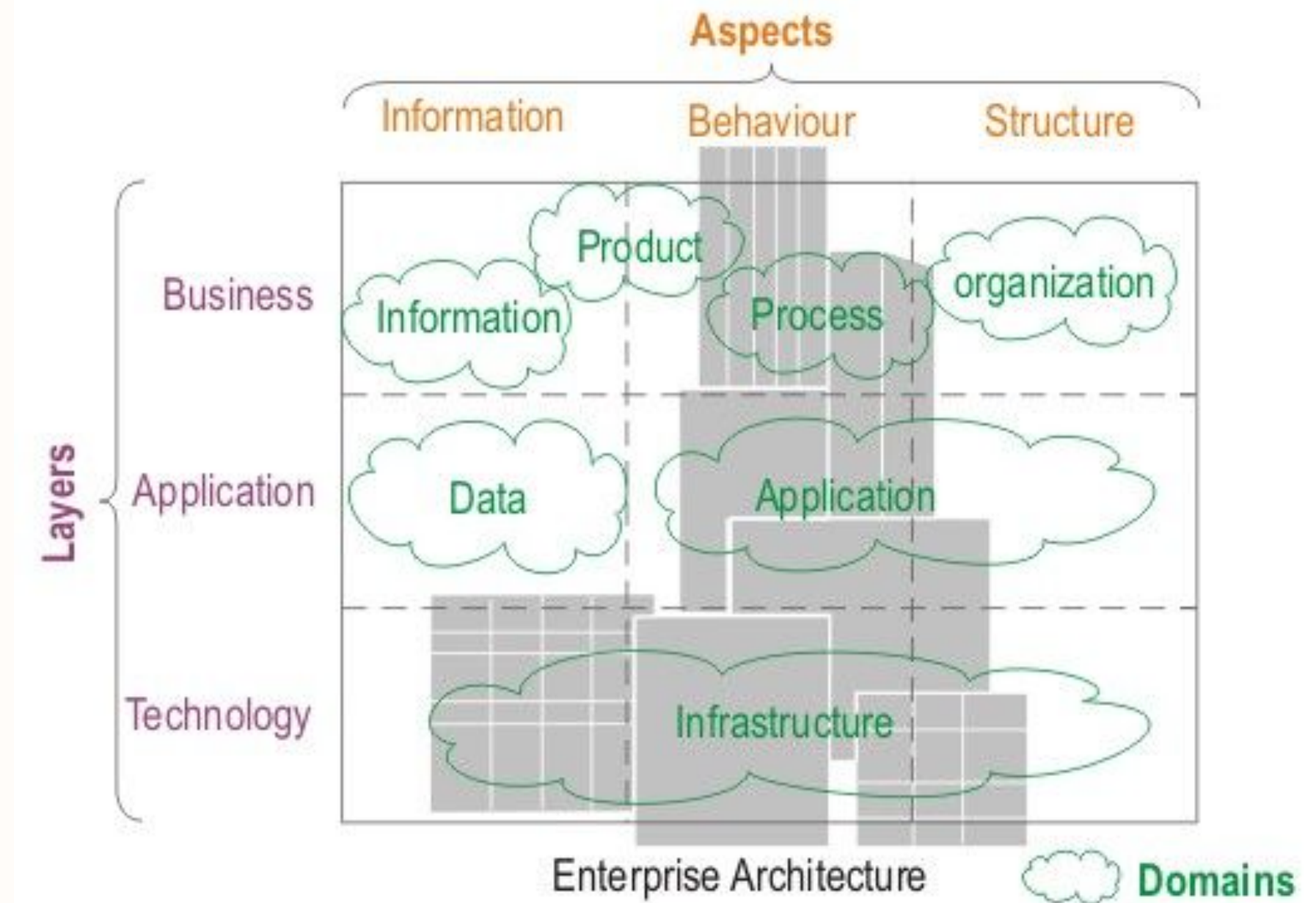
- Enterprise architecture modelling language
- Describes relationships between business, application, and technology

Advantages

- Rich options for description of dependencies and relationships
- Useful for architecture roadmaps
- Standardised

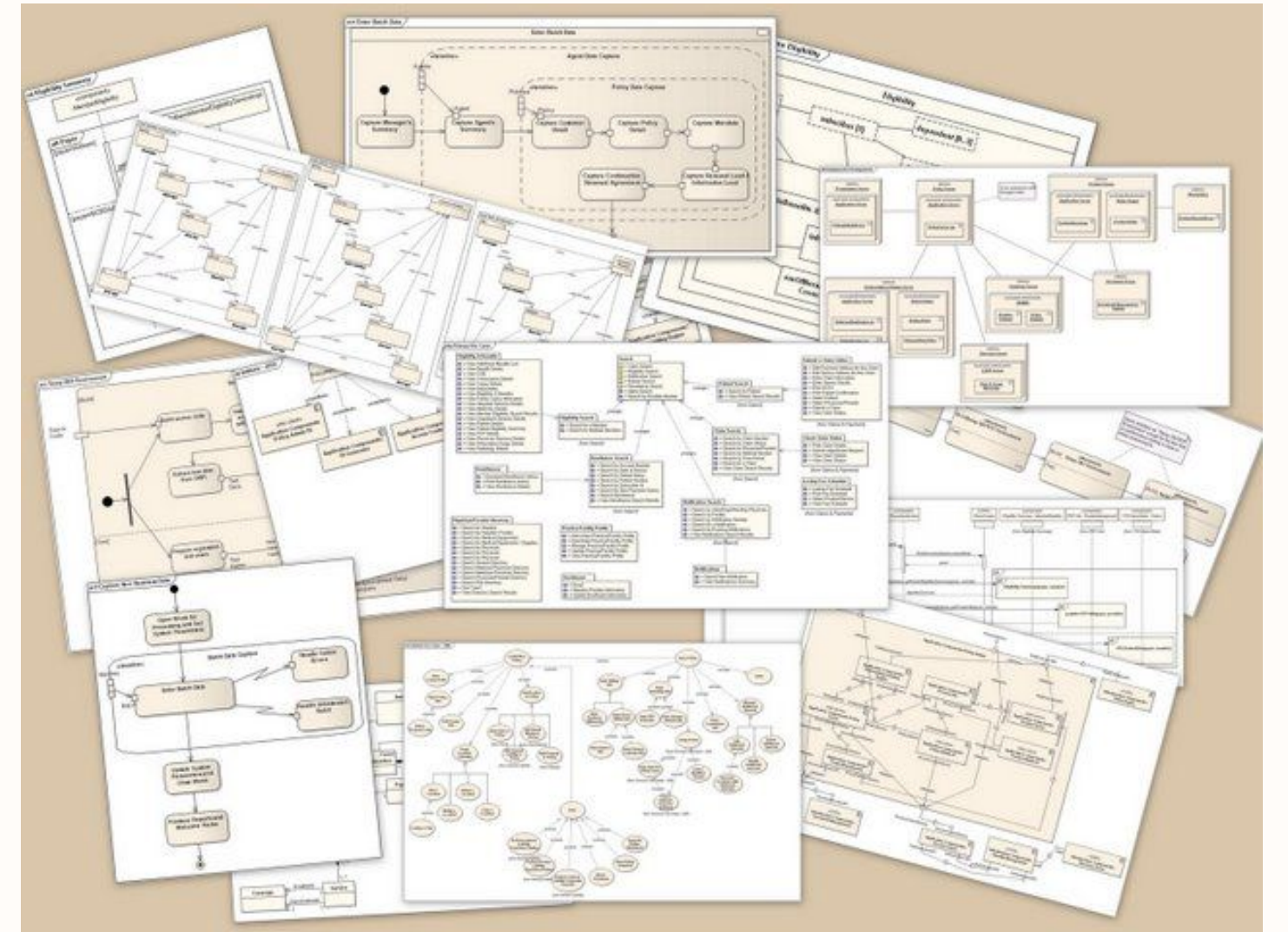
Disadvantages

- Large and complex notation
- Overkill for normal simple systems



Summary

- **Why model?**
 - **Software is complex and abstract**
 - **Models provide simplified views/perspectives**
 - **Different views are suitable for different stakeholders**
 - **Visual models facilitate understanding and communication**
 - **Models are thinking tools**



**A model is not the system — it is a purposeful abstraction of the system.
Choose the models and level of detail according to what you need to
understand and who needs to understand it.**